

Object Oriented Software Engineering

Unit 1 — Introduction to Software Engineering and Modeling through UML

- ✦ Software Engineering Concepts

- ✦ OO Methodologies

- ✦ Definitions & Terminology

- ✦ Object-Oriented Concepts

- ✦ OO Modeling

- ✦ SDLC: Prescriptive & Specialized Models

Topics to be discussed

01	Software Engineering Concepts	Definitions, goals, layers, and why disciplined engineering matters
02	Object-Oriented Concepts	Objects, classes, and the four pillars of OOP with examples
03	OO Methodologies	Booch, OMT, OOSE and how they converged into UML
04	OO Modeling	How analysis and design models are built step by step
05	Definitions & Terminology	A working glossary of core OO and SE terms
06	UML in Practice	Use case and class diagrams for a Library Management System
07	SDLC Process Models	Prescriptive models (Waterfall, Incremental, Spiral, V-Model) and Specialized models (Agile, Component-based, Formal Methods)

What is Software Engineering?

IEEE Definition: The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software.

In simple terms

- ✦ It is engineering discipline applied to software — planning, building and maintaining it the way a bridge or building is engineered, not just ‘coded’.
- ✦ It covers the full journey: requirements → design → coding → testing → deployment → maintenance.
- ✦ Goal: deliver software that is correct, reliable, on time, within budget, and easy to change later.

Why It Matters for AI Products

- ✦ An AI model is only one component of a real product — APIs, databases, security and deployment surround it.
- ✦ Poor architecture, weak testing or bad deployment can break an excellent model.
- ✦ Software Engineering gives AI-specialisation students the discipline to ship AI reliably, not just train it.

The Layers of Software Engineering

Every mature engineering discipline rests on a layered foundation — software engineering is no different.

Quality Focus

The bedrock commitment to building software that genuinely meets user needs and standards.

Process

The framework that defines how activities are organized, managed and tracked from start to finish.

Methods

The technical how-to — requirements analysis, design, coding and testing techniques.

Tools

Automated or semi-automated support (IDEs, UML tools, CI/CD, testing frameworks) for methods and process.

Program vs. Software Product

A Program

- ✦ Written by an individual for personal or narrow use.
- ✦ Little to no documentation.
- ✦ Rarely tested against varied real-world inputs.
- ✦ Hard for anyone but the author to maintain.

A Software Product / System

- ✦ Built by teams, for a wide base of users, over a long lifetime.
- ✦ Documented, tested, and engineered for change.
- ✦ Meets quality attributes: correctness, reliability, efficiency, usability, maintainability.
- ✦ This gap — program to product — is exactly what Software Engineering bridges.

The Four Pillars of Object Orientation

Object-oriented thinking models software as a set of interacting objects, each combining data and behaviour — much closer to how we see the real world.

1

Encapsulation

Bundling data and the methods that operate on it, hiding internal details behind a clean interface.

2

Abstraction

Exposing only essential features of an object while hiding unnecessary implementation complexity.

3

Inheritance

Letting a new class reuse and extend the attributes and behaviour of an existing class.

4

Polymorphism

Allowing the same operation to behave differently across different classes/objects.

Class and Object — The Blueprint and the Instance

CLASS: Book

Attributes

- title - author - ISBN - status

Methods

+ issue() + returnBook() + renew()

A class is a blueprint — it defines the attributes and methods every object of that type will have, but holds no data of its own.

instantiates
→

Object: b1

title = "Clean Code"
author = "R. Martin"
status = "Issued"

Object: b2

title = "UML Distilled"
author = "M. Fowler"
status = "Available"

Encapsulation & Abstraction — In Practice

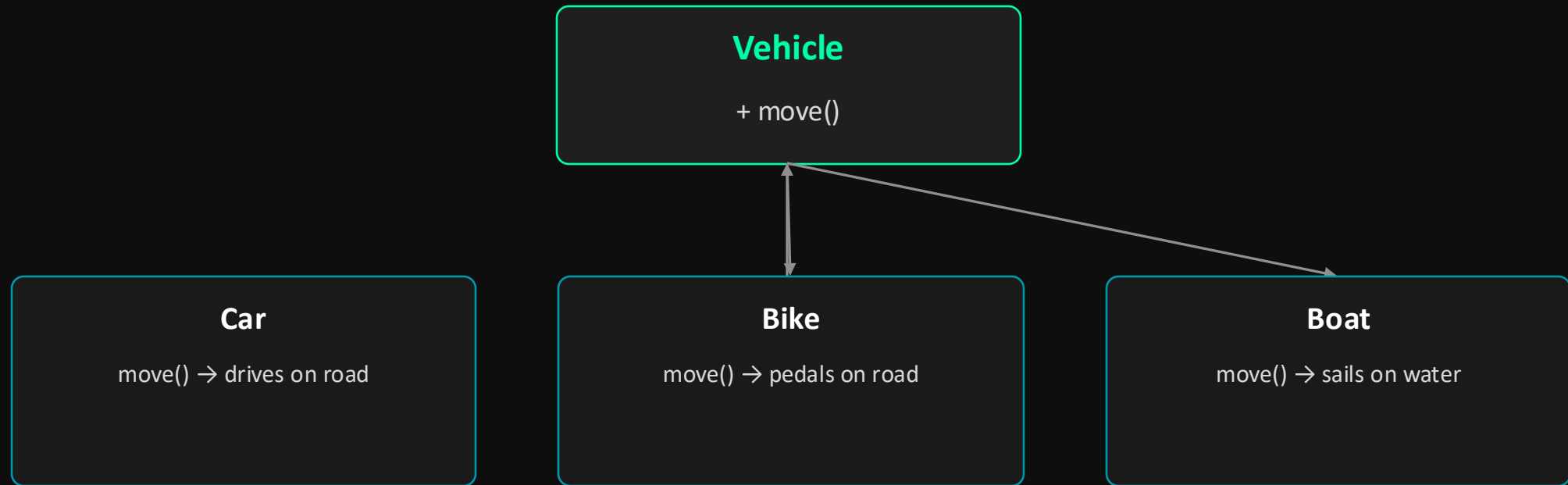
Encapsulation

- ✦ The BankAccount class keeps its balance field private; only its own deposit() and withdraw() methods may change it.
- ✦ Outside code can never set balance directly — it must go through the class's own rules, which protects data integrity.
- ✦ Think of it as a capsule: data and behaviour sealed together, accessible only via a controlled interface.

Abstraction

- ✦ A driver uses accelerate() and brake() without knowing the engine's internal wiring — only the essential interface is visible.
- ✦ In software, an ATM interface hides the internal banking logic and network calls behind two buttons: Withdraw and Balance.
- ✦ Abstraction manages complexity by highlighting what an object does, not how it does it.

Inheritance & Polymorphism — Vehicle Example



- ✦ Inheritance: Car, Bike and Boat all inherit the `move()` operation and any shared attributes from Vehicle, avoiding duplicated code.
- ✦ Polymorphism: the same call `vehicle.move()` produces different behaviour depending on the actual object — drive, pedal or sail.

Why Choose the Object-Oriented Approach?

1

Reusability

Classes and components can be reused across projects, cutting development time.

2

Maintainability

Encapsulated, modular objects are easier to locate, fix and extend safely.

3

Real-World Mapping

Objects mirror real entities (Student, Book, Order), making designs intuitive to reason about.

4

Scalability

New object types can be added via inheritance without disturbing existing code.

5

Data Security

Encapsulation restricts direct access to internal state, reducing accidental corruption.

6

Team Collaboration

Clear class boundaries let teams work on different objects in parallel.

Object-Oriented Methodologies

Before UML existed, several competing methodologies each proposed their own notation for OO analysis and design.

Methodology	Author	Key Contribution
Booch Method	Grady Booch	Rich, detailed notation for design; strong on class & object diagrams; sometimes seen as visually complex.
OMT (Object Modeling Technique)	James Rumbaugh	Emphasises Object, Dynamic and Functional models; strong for analysis of data-heavy systems.
OOSE (Object-Oriented Software Engineering)	Ivar Jacobson	Introduced use cases as the driver of requirements and design — still central in UML today.
Coad-Yourdon	Peter Coad & Ed Yourdon	Simple, easy-to-learn notation focused on OOA; good for smaller systems.

How the Methodologies Converged into UML



1994–95 — The ‘Three Amigos’ unite

Booch and Rumbaugh join Rational; they begin merging the Booch method and

1995 — Jacobson joins

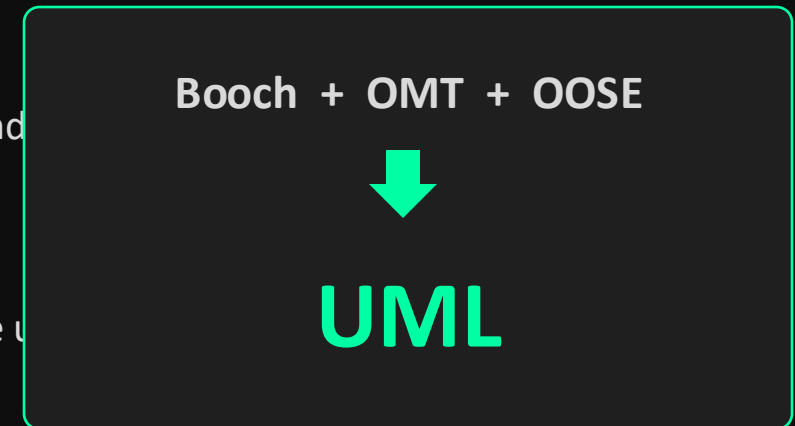
Ivar Jacobson (OOSE) joins Rational; use-case driven analysis is folded into the u

1996–97 — Unified Modeling Language

Their combined notation is published and submitted to the OMG as UML 1.0, standardising OO modeling.

1997–Today — Industry Standard

UML becomes the common visual language for OO analysis, design and documentation across the industry.



What is Object-Oriented Modeling?

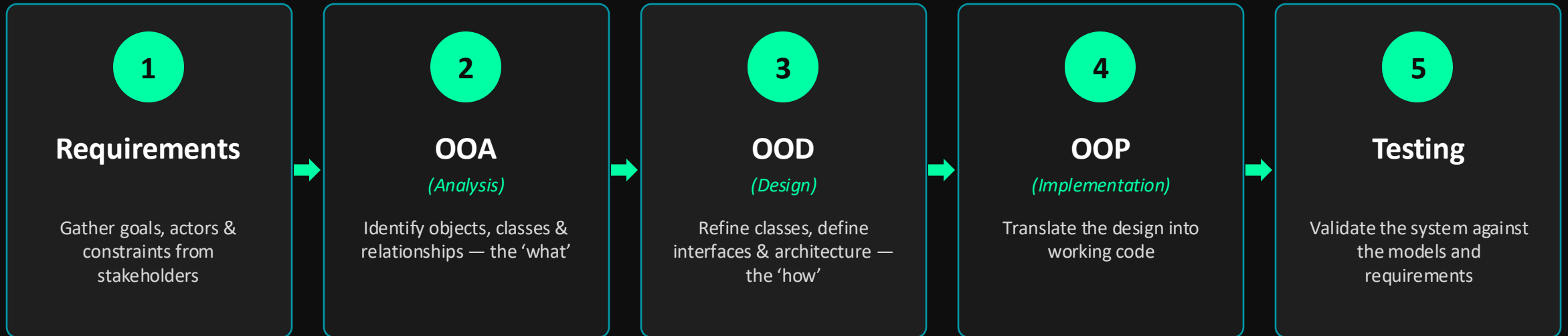
Definition

- ✦ OO Modeling is the process of representing a real-world system as a set of interacting objects, using UML diagrams, before any code is written.
- ✦ It captures both structure (what the system is made of) and behaviour (how it acts over time).
- ✦ A model is an abstraction — it deliberately omits detail irrelevant to the current purpose.

Why We Model

- ✦ Communication: gives clients, testers and developers one shared, visual language.
- ✦ Early error detection: flaws surface on a diagram, long before they cost time in code.
- ✦ Documentation: the model outlives the original team and eases future maintenance.
- ✦ Blueprint for construction: guides coding, just as an architect's plan guides a building.

The OO Modeling Process



- ✦ The process is iterative: models are validated with stakeholders and refined — it is rarely a single straight pass.
- ✦ OOA answers 'what should the system do'; OOD answers 'how will the system do it'.

Core Terminology — I

Object

A concrete entity with state (attributes) and behaviour (methods), e.g. a specific book on a shelf.

Class

A blueprint/template that defines the attributes and methods common to a set of objects.

Attribute

A data value held by an object, describing one aspect of its state (e.g. title, price).

Method

An operation/function defined in a class that an object can perform.

Message Passing

The mechanism by which objects communicate — one object invokes a method on another.

Interface

A contract listing operations a class promises to provide, without specifying implementation.

Core Terminology — II

Association

A general structural relationship indicating objects of one class are connected to another (e.g. Student – Course).

Aggregation

A 'has-a' whole-part relationship where the part can exist independently of the whole (Library has Books).

Composition

A stronger 'owns-a' relationship where the part's lifetime depends on the whole (House owns Rooms).

Coupling

The degree of interdependence between classes/modules — lower coupling eases maintenance.

Cohesion

How closely the responsibilities within a single class are related — higher cohesion is desirable.

Multiplicity

The number of instances of one class that relate to one instance of another (e.g. 1..*).

UML — The Modeling Language We'll Use

Why UML

- ✦ Unified Modeling Language: a standard visual notation, maintained by the OMG, for specifying and documenting OO systems.
- ✦ Used across analysis, design, documentation and team communication.
- ✦ Diagram-based, so both technical and non-technical stakeholders can follow it.

Structural

Class, Object, Component, Deployment, Package — what the system is made of.

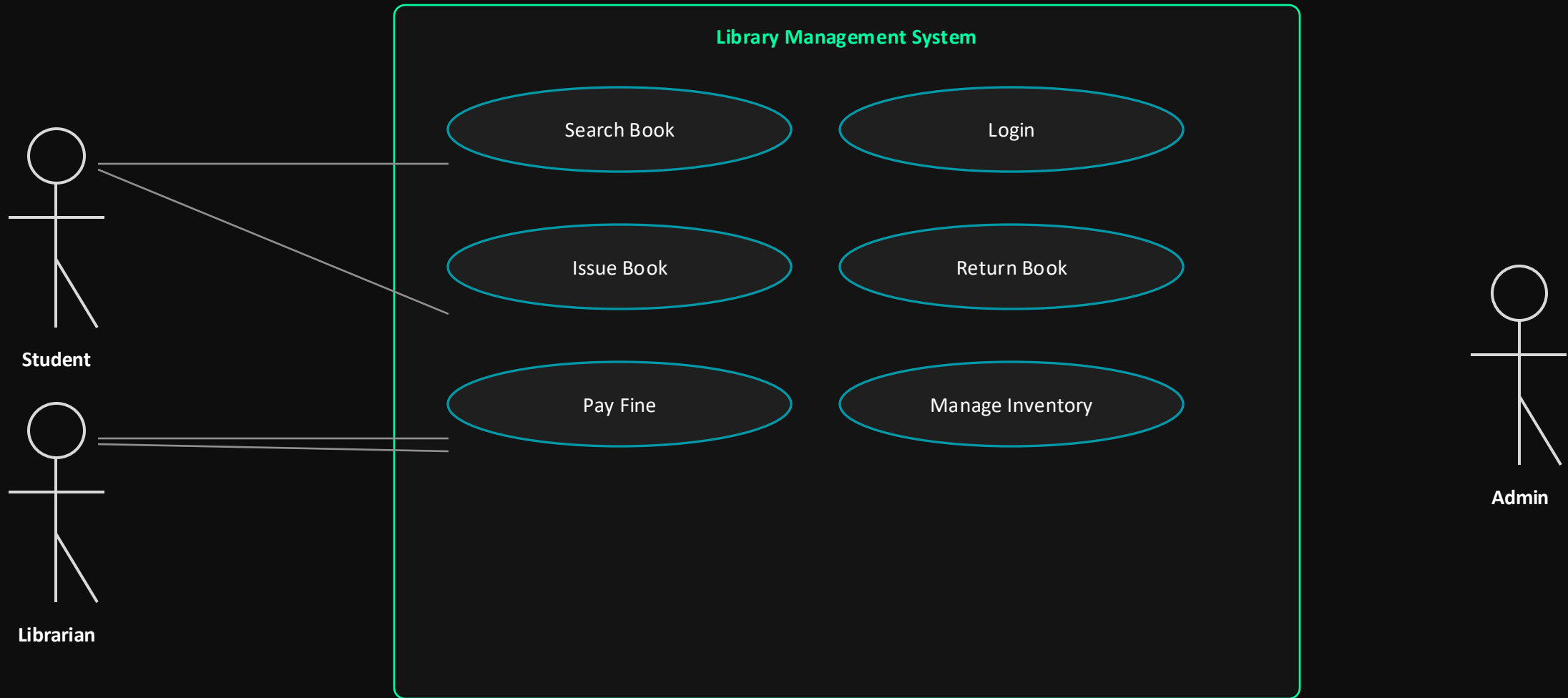
Behavioral

Use Case, Activity, State Machine — how the system behaves.

Interaction

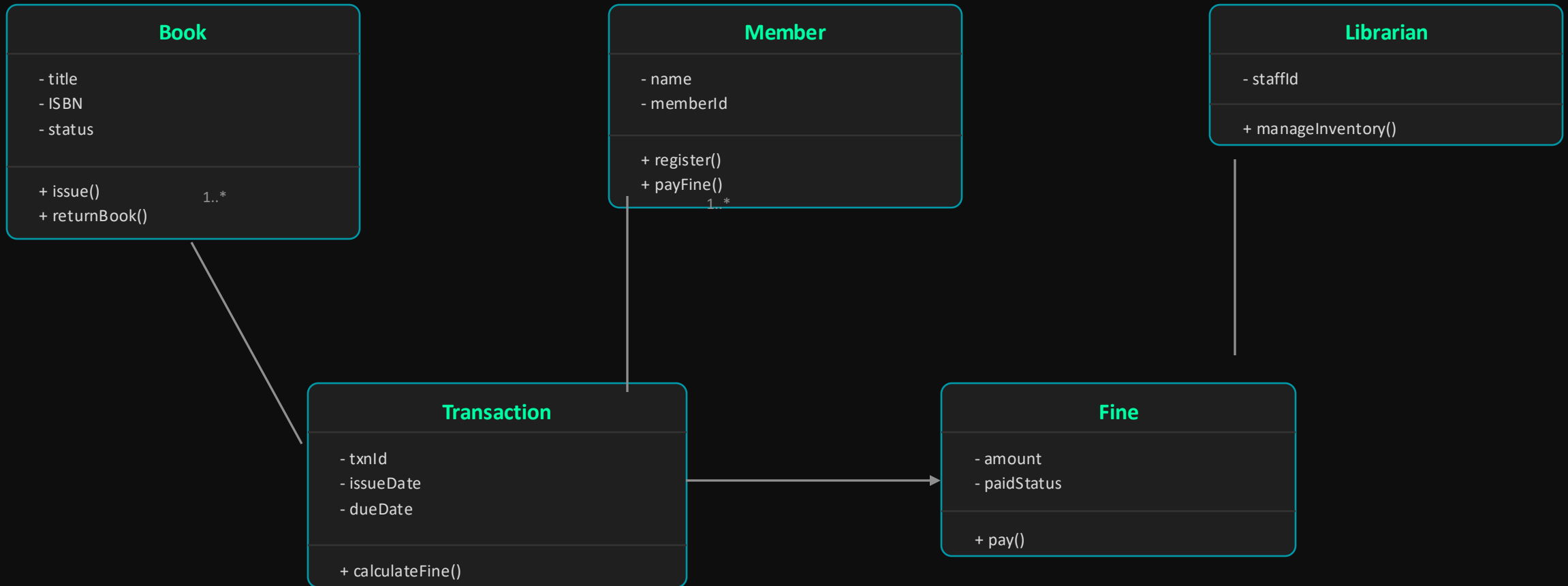
Sequence, Communication, Timing — how objects exchange messages.

Use Case Diagram — Library Management System



Actors (Student, Librarian, Admin) trigger use cases; the boundary box shows what belongs inside the system's scope.

Class Diagram — Library Management System



Book – Transaction – Member show association; Transaction – Fine models the fine generated on a late return.

Software Process Models — Why We Need Them

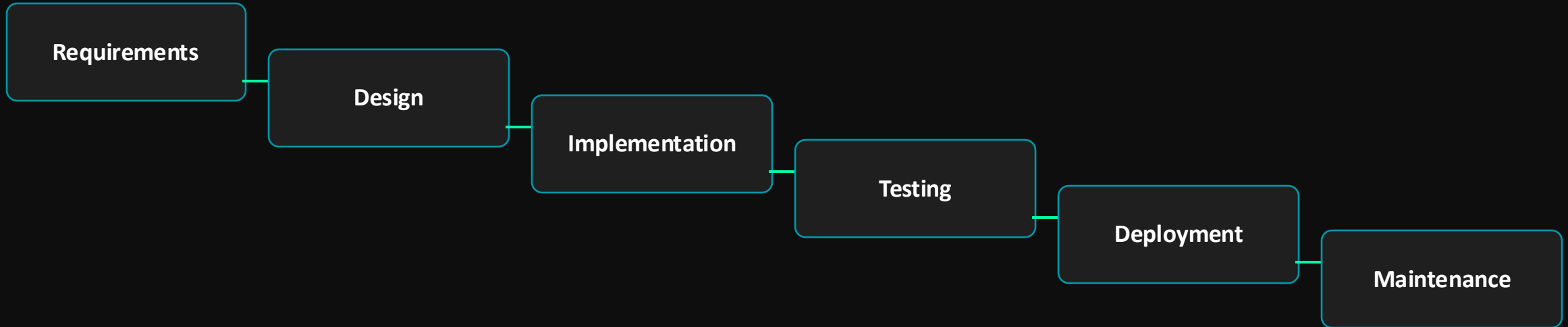
What is a Process Model?

- ✦ A structured roadmap describing the order and manner in which SDLC activities — requirements, design, coding, testing, deployment — are carried out.
- ✦ It sets expectations for planning, resourcing and tracking a software project.
- ✦ Different projects carry different risk, scale and stability of requirements — no single model fits all.

Two Broad Families

- ✦ Prescriptive Process Models — traditional, structured models such as Waterfall, Incremental, RAD, Spiral and the V-Model.
- ✦ Specialized Process Models — models built for specific needs: Component-Based Development, Formal Methods, Agile, and Concurrent Development.

Waterfall Model



Key Points

- ✦ Strictly sequential: each phase must complete before the next begins; the model ‘flows downhill’ like a waterfall.
- ✦ Best suited when requirements are clear, stable and unlikely to change during development.
- ✦ Drawback: changes discovered late (e.g. in testing) are expensive to trace back and fix, since there is little iteration.

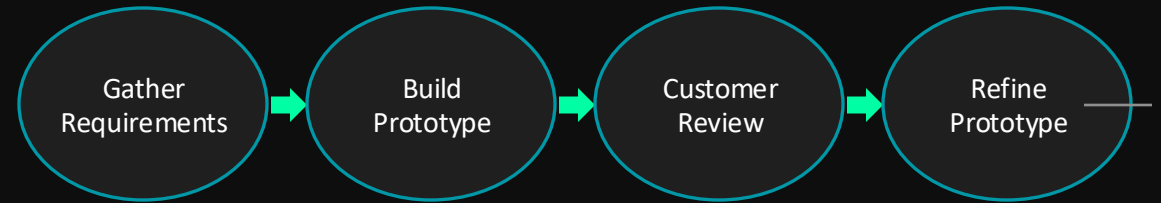
Incremental Model & Prototyping Model

Incremental Model



- ✦ Delivers the system in small, usable increments; each adds functionality to the previous one.
- ✦ Earlier increments give users working software sooner and allow feedback to shape later increments.
- ✦ Reduces risk versus Waterfall since problems surface earlier, increment by increment.

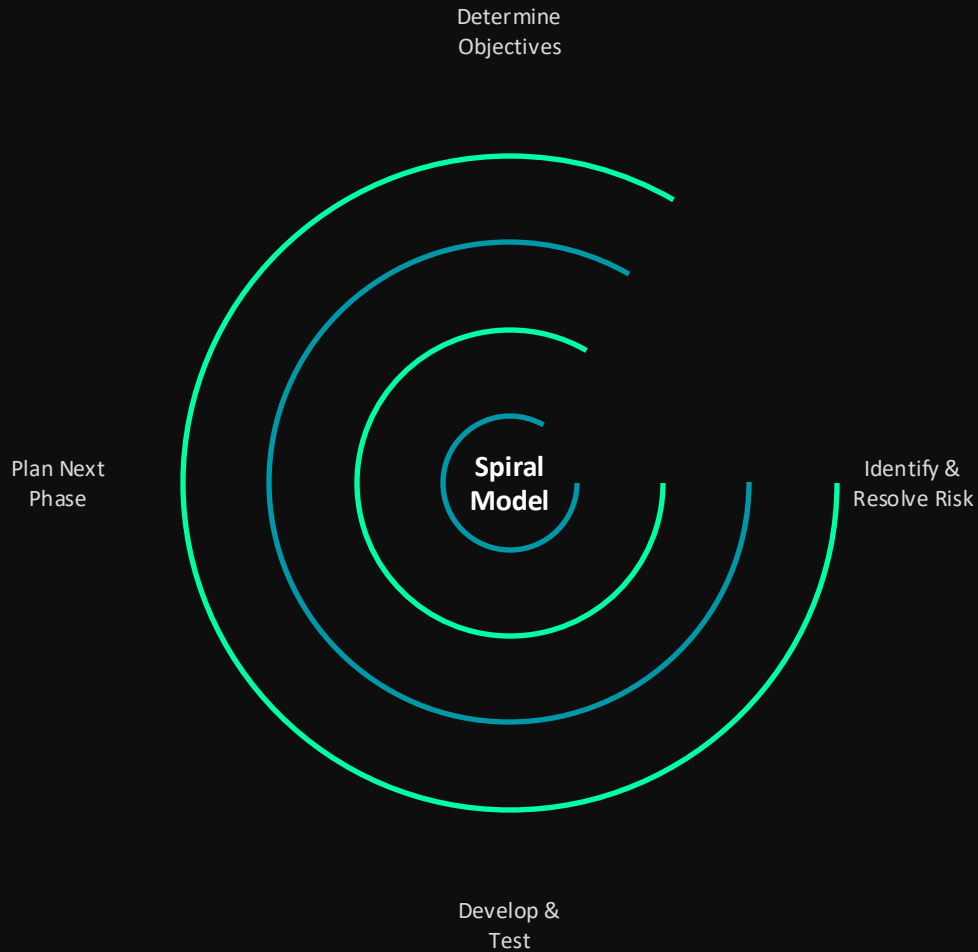
Prototyping Model



*repeat until
accepted*

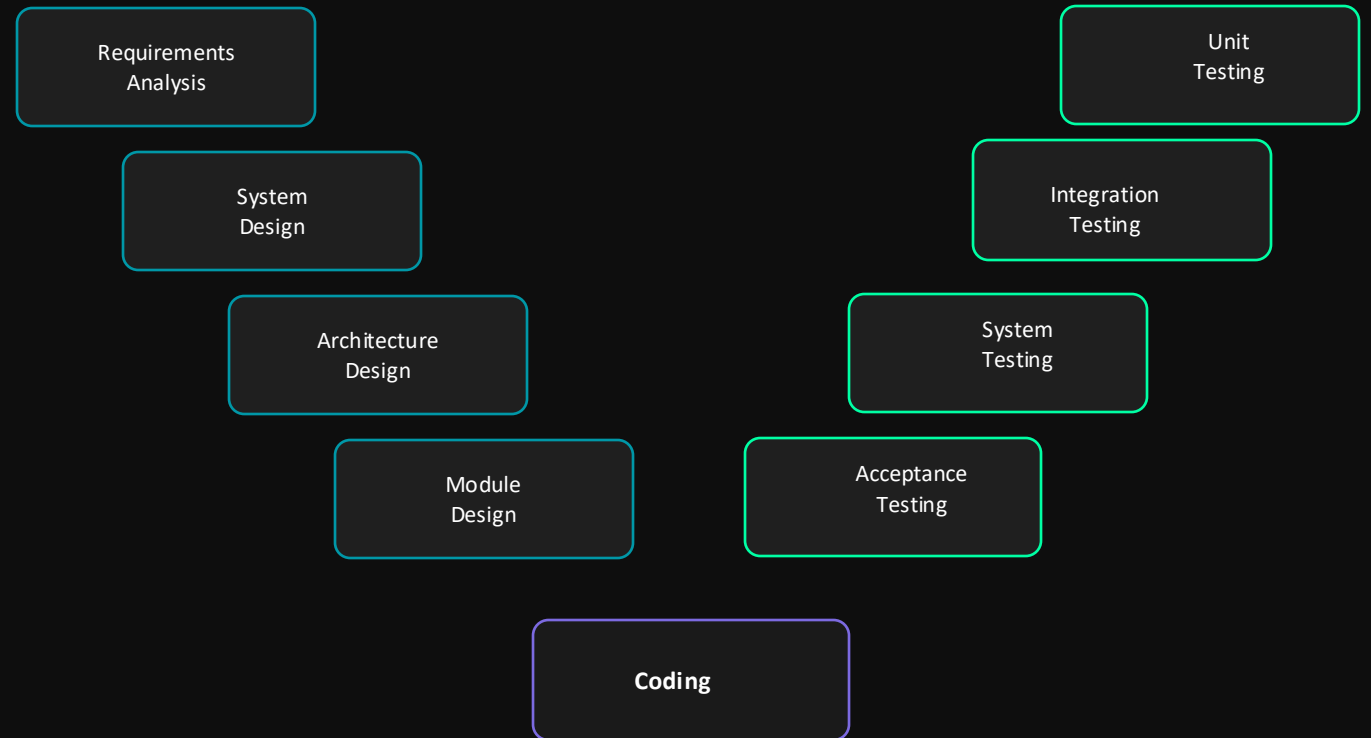
- ✦ Builds a working mock-up quickly so users can see and react to the system before full development.
- ✦ Ideal when requirements are unclear or evolving — the prototype clarifies what is really needed.
- ✦ The final prototype (or lessons from it) feeds into the actual system's requirements and design.

Spiral Model & V-Model



- ◆ Combines iterative prototyping with Waterfall's discipline; each loop assesses risk before proceeding — suited to large, risky projects.

V-Model



- ◆ Each development phase (left) pairs with a matching test phase (right) — verification and validation are planned from day one.

Specialized Process Models

1

Component-Based Development

Assembles the system from pre-built, reusable software components rather than coding from scratch.

Reuse-heavy systems, e.g. enterprise apps with common modules

2

Formal Methods

Uses mathematically rigorous specification and verification techniques to prove correctness.

Safety-critical systems: avionics, medical devices, defense

3

Agile (Scrum / XP)

Delivers working software in short iterations (sprints), embracing changing requirements throughout.

Fast-changing products, startups, evolving requirements

4

Concurrent Development

Models activities (analysis, design, coding) as parallel states rather than a strict sequence.

Teams needing overlapping, simultaneous work across activities

WRAP-UP

Choosing the Right Process Model

Requirements are clear & stable

Waterfall

Requirements are unclear or evolving

Prototyping

Need to deliver value early, in stages

Incremental

Large, high-risk project

Spiral

Testing must be planned from day one

V-Model

Requirements change constantly

Agile

System correctness is safety-critical

**Formal
Methods**

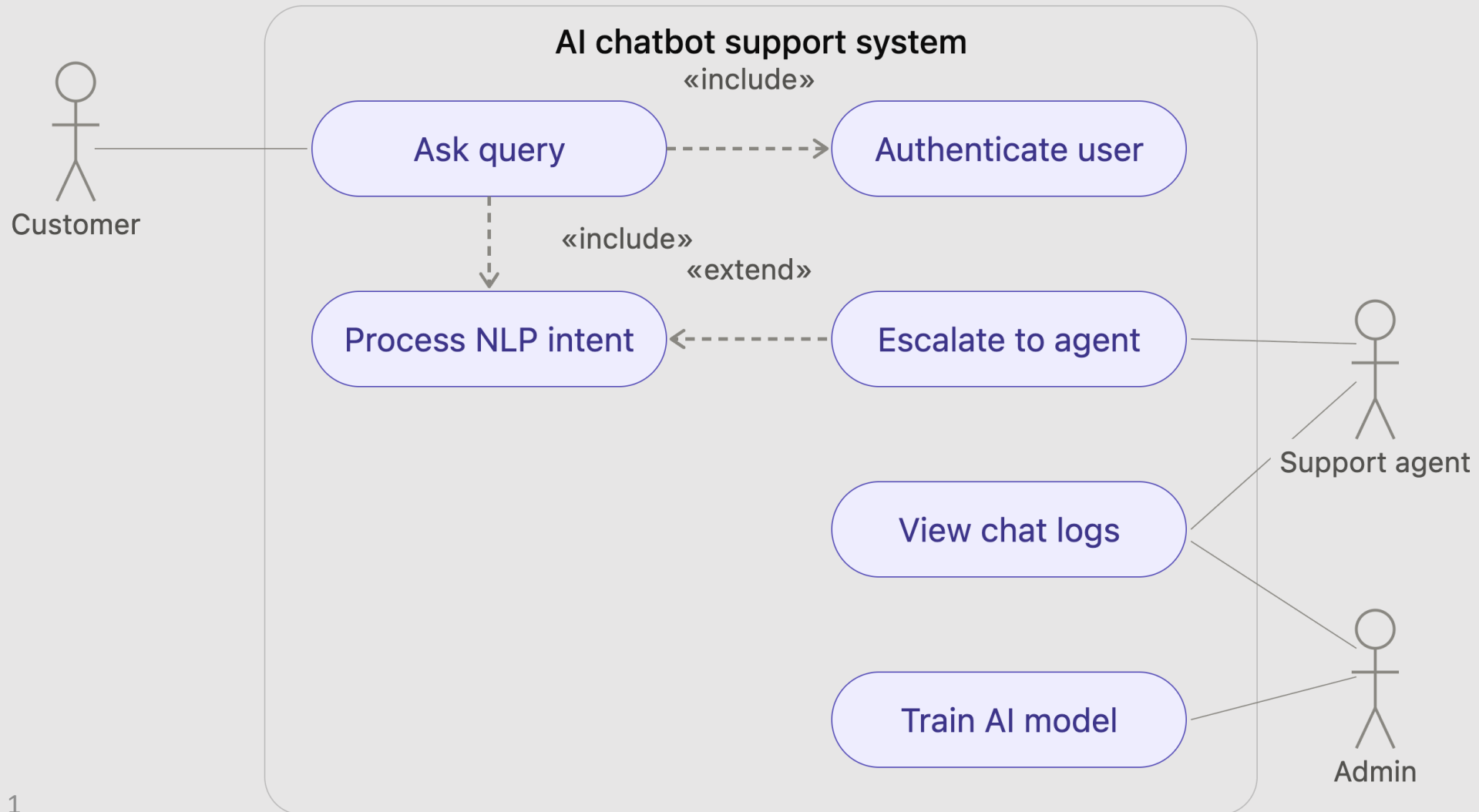
Reusable modules already exist

**Component-
Based**

Use Case Diagram

- **Actor** — a stick figure representing a user or external system (human or another system, e.g. a payment gateway or an AI engine).
- **Use case** — an oval/ellipse naming a discrete piece of functionality the system offers.
- **System boundary** — a rectangle enclosing all use cases, labelled with the system name.
- **Association** — a plain line between actor and use case.
- **«include»** — a dashed arrow from a base use case to one it always triggers (mandatory sub-behaviour), e.g. "Ask query" always includes "Authenticate user".
- **«extend»** — a dashed arrow from an optional/conditional use case to the one it extends, e.g. "Escalate to agent" extends "Process NLP intent" only when confidence is low.
- **Generalization** — a solid line with a hollow triangle, used when one actor or use case is a specialization of another (e.g. "Registered user" and "Guest user" generalize from "User").

Use Case Diagram



Use Case Diagram - Examples

1. AI-based resume screening system

- Actors: Job applicant, HR recruiter, AI ranking engine, Admin
- Use cases: Upload resume → *includes* Extract skills (NLP parsing) → *includes* Match job description; Rank candidates; Shortlist applicant (*extends* Rank candidates, only if score > threshold); Schedule interview; Retrain ranking model

2. AI-based movie/product recommendation system

- Actors: User, Recommendation engine, Admin
- Use cases: Browse catalog; View recommendations → *includes* Analyze user behavior, *includes* Generate similarity score; Rate item; Update user profile (*extends* Rate item); Manage catalog

3. AI-based fraud detection system for banking

- Actors: Customer, Bank employee, AI fraud engine, Regulatory system
- Use cases: Initiate transaction → *includes* Validate transaction, *includes* Compute risk score; Flag suspicious transaction (*extends* Compute risk score, only if score exceeds threshold); Review flagged case; Report to regulator; Retrain fraud model

Key Takeaways

From Concepts to Models to Process

- ✦ Software Engineering brings discipline to building reliable, maintainable systems — essential even (especially) around AI components.
- ✦ The four OO pillars — Encapsulation, Abstraction, Inheritance, Polymorphism — shape how we design classes and objects.
- ✦ OO methodologies converged into UML, the shared visual language for OO analysis and design.
- ✦ SDLC process models — prescriptive and specialized — must be matched to a project's stability, risk and pace of change.